

第二天 變數與運算子

大里 APCS 營隊 · 資料型態、運算子與流程控制

郭家睿

今天的目標

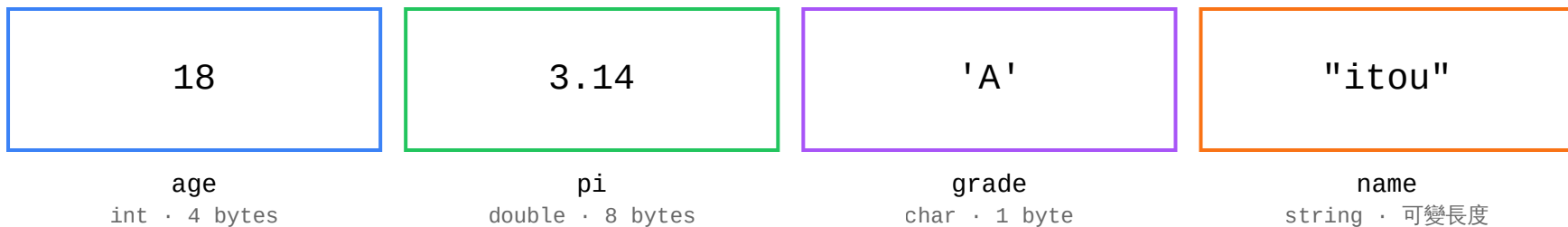
- 認識基本資料型態：``int`` / ``double`` / ``char`` / ``bool`` / ``string``
- 熟悉各種運算子：算術、遞增遞減、比較、邏輯
- 用 **if / else if / else** 做流程控制

回顧昨天

- 程式要先編譯才能執行；編譯期錯誤 vs 執行期錯誤
- ``cout <<`` 輸出、``cin >>`` 輸入，箭頭指向誰資料就流向誰
- 所有程式都是 輸入 → 處理 → 輸出

資料型態

變數就是一個有型態的盒子



型態告訴電腦：這個盒子裝什麼，以及裡面的資料該怎麼理解。

- 記憶體裡只有 0 和 1，本身沒有意義
- 型態決定資料需要多少空間
- 型態決定 0 和 1 應該如何解讀

基本資料型態

型別	說明	範例值	大約範圍
<code>`int`</code>	整數	<code>`42`</code> , <code>`-7`</code>	約 ± 21 億
<code>`double`</code>	浮點數	<code>`3.14159`</code>	約 15 位有效數字
<code>`char`</code>	單一字元	<code>`'A'`</code> , <code>`'9'`</code>	一個字元
<code>`bool`</code>	布林值	<code>`true`</code> , <code>`false`</code>	只有兩種值
<code>`string`</code>	字串	<code>`"Hello"`</code>	任意長度文字

小數一律用 ``double`` , 不要用 ``float`` —— ``float`` 只有約 7 位有效數字, 很容易算錯。

`int` : 整數

```
int age = 16;  
int score = -5;    // 可以是負的  
int count = 0;
```

沒有小數點的數字都用 `int`。範圍大約 ± 21 億，今天的題目都在這個範圍內，之後遇到更大的數字才需要換成 `long long`（今天先不用）。

`double`：浮點數

```
double pi = 3.14159;  
double average = 87.5;  
double temperature = -3.2;    // 也可以是負的
```

只要牽涉到小數點，一律用 ``double``。等一下會看到，``int`` 除以 ``int`` 是會把小數捨去的，這是今天最重要的踩雷點之一。

`char` : 單一字元

```
char grade = 'A';  
char op = '+';
```

`char` 只能裝一個字元，而且要用單引號 `' '`，不是雙引號。 `'A'` 是 `char`，`"A"` 是只有一個字的 `string`，兩者不一樣。

`bool`：布林值

```
bool isStudent = true;  
bool isRaining = false;
```

只有 `true``（成立）和 `false``（不成立）兩種值，通常用來存「條件成不成立」，今天的 if 判斷式算出來的結果，本質上就是一個 `bool``。

``string``：字串

```
string name = "itouSouta";  
string city = "Taichung";
```

用雙引號 `" "` 包起來的一串文字。今天不會深入 ``string`` 的細節，先會宣告、讀取、輸出就夠用，第四天會再深入。

float 與 double 的差異

```
float a = 0.1234567890123456;  
double b = 0.1234567890123456;  
  
cout.precision(17);  
cout << a << endl; // 0.12345679104328156 ← 後面開始不準  
cout << b << endl; // 0.12345678901234560 ← 比較準
```

「float」只有約 7 位有效數字，「double」約 15 位。**記不住就用「double」，今天以後你寫的每一個小數都建議用「double」。

變數宣告與初始化

```
int a;           // 宣告：先佔位，值不確定（垃圾值）
a = 10;          // 賦值：把值放進去

int b = 20;      // 宣告同時初始化（建議！）
int x = 1, y = 2, z = 3;  // 一次宣告多個
```

養成「**宣告就初始化**」的習慣。沒初始化的變數裡面是上一個程式留下的垃圾，在你的電腦上剛好是 0，在判題機上可能是 -858993460。

常見錯誤：忘記初始化就使用

```
int sum;  
cout << sum << endl;    // 忘記先給值，印出垃圾值
```

這種錯誤**編譯不會報錯**，程式能跑，但答案是隨機的垃圾值。是最難抓的一種錯誤，因為看起來「有印出東西」，容易被忽略。

練習時間

第 2 題 班級點名卡

輸入一位學生的姓名、座號、分組代號，依格式印出點名卡。

輸入 3 行：姓名（字串）、座號（整數）、分組代號（字元）

輸出 一行：`座號 號 姓名 (分組 組)`

輸入

Amy

12

B

輸出

12 號 Amy (B 組)

這題還不需要任何運算子，重點是**型態選對**：三個資料剛好對應 `string`、`int`、`char` 三種型態。

第 2 題 讀題

- 輸入：三行，依序是姓名（字串）、座號（整數）、分組代號（單一大寫字母）
- 輸出：一行，把三個值套進固定格式
- 處理：不用算任何東西，型態讀對、格式排對就結束了

第 2 題 完整程式

```
#include <iostream>
using namespace std;

int main() {
    string name;
    int seat;
    char group;
    cin >> name >> seat >> group;
    cout << seat << " 號 " << name << " (" << group << " 組) " << endl;
    return 0;
}
```

`cin >> a >> b >> c` 可以一次讀進不同型態的變數，昨天已經看過；這裡刻意混用 `string`、`int`、`char` 三種型態，練習「看到資料就反應出型態」的直覺。輸出的括號是全形「()」，不是英文鍵盤打的半形「()」，判題逐字比對，打錯就 WA。

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 2 題

寫到 AC 為止，再往下聽

第 3 題 攝氏轉華氏

輸入攝氏溫度 C，換算成華氏溫度 F 輸出。

$$F = C \times 9 \div 5 + 32$$

輸入

37

輸出

98.6

C 可能有小數點，公式裡也有除法——這題練的是 `double` 型態，加上加、減、乘、除幾個最基本的算術運算子，還不會碰到比較或邏輯運算子。

第 3 題 讀題

- **輸入**：一個實數 C ，代表攝氏溫度，可能有小數點
- **輸出**：換算後的華氏溫度 F
- **處理**：套公式 `F = C * 9 / 5 + 32`，直接算完印出來

C 可能不是整數（像 36.5），要宣告成 `double` 才讀得進去；如果宣告成 `int`，遇到小數點的輸入會讀壞。

第 3 題 完整程式

```
#include <iostream>
using namespace std;

int main() {
    double c;
    cin >> c;
    double f = c * 9 / 5 + 32;
    cout << f << endl;
    return 0;
}
```

``c``、``9``、``5``、``32`` 混在一起算，只要其中一個是 ``double``（這裡是 ``c``），整條算式就會自動用小數計算，不用每個數字都額外轉型。

如果算出來的 F 剛好沒有小數（例如 $C = 0$ 時 $F = 32$ ），``cout`` 印 ``double`` 預設會直接印 ``32``，不會多印一個 ``.0``——不用懷疑自己哪裡算錯，這就是正確答案。

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 3 題

寫到 AC 為止，再往下聽

運算子基礎

算術運算子

```
int a = 17, b = 5;

cout << a + b;    // 22
cout << a - b;    // 12
cout << a * b;    // 85
cout << a / b;    // 3    (整數除法，捨去小數)
cout << a % b;    // 2    (餘數 modulo)
```

`+` - `\*` 跟數學一樣好懂，`/` 等一下會講陷阱，`%` 是今天的新面孔。

`%` 餘數：怎麼算出來的

17 % 5

17 除以 5，商是 3、餘數是 2（因為 $5 \times 3 = 15$ ， $17 - 15 = 2$ ）。`%` 拿到的就是這個餘數。

10 % 3
1

9 % 3
0

7 % 7
0

3 % 7
3

``%`` 的兩個常見用途

- 判斷整除 / 倍數：``n % 3 == 0`` 代表 n 是 3 的倍數（餘數是 0）
- 判斷奇偶數：``n % 2 == 0`` 是偶數，``n % 2 != 0`` 是奇數
- 取出末位數字：``n % 10`` 拿到 n 的個位數（明天翻轉數字會用到）

小測驗：`%` 的計算

`23 % 5` 等於多少？

「3」——`23 = 5 × 4 + 3`，餘數是 3。

if 基礎

if 是什麼

- `if` 讓程式檢查一個條件，成立才執行接下來的程式
- 條件寫在 `if` 後面的括號裡，結果一定是 `true` / `false`
- 只有條件成立 (true) 時，大括號裡的内容才會被執行

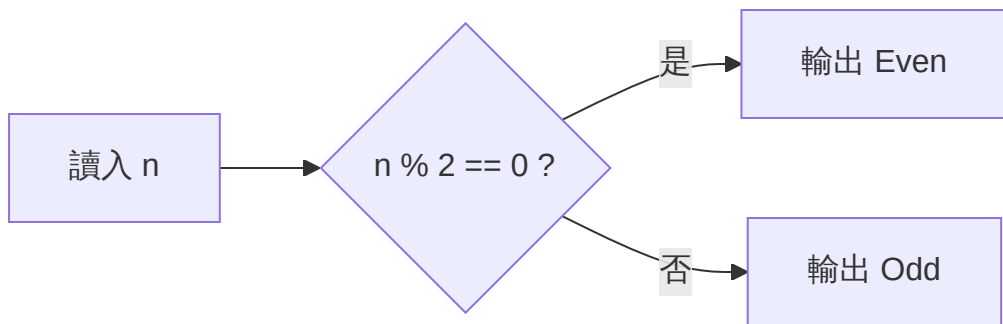
只有 if，沒有 else 會怎樣

```
int score = 45;

if (score < 60) {
    cout << "不及格" << endl;
}
cout << "考試結束" << endl;
```

沒有 `else` 也完全合法。條件不成立的話，就直接跳過大括號，往下繼續執行 `if` 之後的程式碼。

if / else : 讓程式做決定



```
if (n % 2 == 0) cout << "Even" << endl;  
else          cout << "Odd"  << endl;
```


else 什麼時候會觸發

- ``else`` 只有在 ``if`` 的條件**不成立**時才會執行
- 一個 ``if`` 最多只能配一個 ``else``
- ``if`` 和 ``else`` 是**互斥**的：兩者當中一定剛好執行一個

練習時間

第 4 題 判斷奇偶數

輸入一個整數，判斷奇數還是偶數。

輸入 一行一個整數 n ($-10^9 \leq n \leq 10^9$)

輸出 偶數輸出 ``Even``，否則輸出 ``Odd``

輸入

7

輸出

Odd

剛好用到你剛學的兩樣東西：``%`` 運算子和 ``if / else``。

第 4 題 讀題

- 輸入：一個整數，可能是負數
- 輸出：`Even` 或 `Odd`，注意大小寫
- 處理：只需要判斷一次，用 `%` 檢查能不能被 2 整除

第 4 題 規劃程式骨架

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    // 這裡判斷奇偶，輸出結果
    return 0;
}
```

第 4 題 完成判斷

```
if (n % 2 == 0) cout << "Even" << endl;  
else          cout << "Odd" << endl;
```

負數陷阱： `-7 % 2` 在 C++ 是 `-1`，不是數學上習慣的 `1`。如果寫成 `n % 2 == 1` 判斷奇數，負的奇數會判斷失敗。永遠用 `n % 2 == 0` 判斷偶數，其餘情況自然就是奇數，比較安全。

第 4 題 完整程式

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (n % 2 == 0) cout << "Even" << endl;
    else          cout << "Odd" << endl;
    return 0;
}
```

注意大小寫：是 `Even` / `Odd`，不是 `even` / `odd`。判題逐字比對。

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 4 題

寫到 AC 為止，再往下聽

if 重複使用的技巧

先假設，再逐一比較更新

- 先假設第一個數字最大，記在一個變數裡
- 拿第二個數字跟目前記的最大值比，比較大就換掉
- 拿第三個數字再比一次
- 走完所有數字，記下來的就是真正的最大值

注意：這裡是**兩個獨立的 if**，不是 if/else——因為每一次比較都是「獨立事件」，跟前一次比較的結果無關。

第 5 題 三數取最大值

輸入三個整數，輸出最大的一個。

輸入

3 7 5

輸出

7

第 5 題 逐步追蹤

輸入 ``3 7 5`` 時：

步驟	動作	ans
開始	假設 $a=3$ 最大	3
檢查 $b=7$	$7 > 3$ ，換成 7	7
檢查 $c=5$	$5 > 7$ ？不成立，不換	7

走完三個數字，``ans`` 是 ``7``，就是答案。

第 5 題 程式

```
int a, b, c;
cin >> a >> b >> c;

int ans = a;           // 先假設 a 最大
if (b > ans) ans = b;   // b 更大就換成 b
if (c > ans) ans = c;   // c 更大就換成 c

cout << ans << endl;
```

這個「**先假設，再逐一比較更新**」的招式，第四天找陣列最大值時會原封不動再用一次。

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 5 題

寫到 AC 為止，再往下聽

中場休息

15 分鐘後回來，我們進入運算子與流程控制的進階內容

運算子進階

遞增與遞減

```
int n = 10;
```

```
n++;      // n = 11    等同 n = n + 1
```

```
n--;      // n = 10    等同 n = n - 1
```

`++` 讓變數自己加 1，`--` 讓變數自己減 1。這是「複合指定運算子」的一種，之後寫迴圈時（明天）會天天用到。

前置 vs 後置：`++n` 跟 `n++` 差在哪

```
int n = 5;  
cout << n++;    // 印出 5，之後 n 變成 6 (先用舊值，再加)  
cout << ++n;    // 印出 7，因為先加，n 變成 7 再印
```

差別只在「單獨一行使用」時看不出來，兩者都會讓 n 加 1。差別只有在跟其他運算式混用、當下就要用到那個值的時候才會顯現。今天先知道兩種寫法都存在，之後看到 `++n` 不要慌。

複合指定運算子

```
int n = 10;
```

```
n += 5; // n = n + 5 → 15
```

```
n -= 3; // n = n - 3 → 12
```

```
n *= 2; // n = n * 2 → 24
```

```
n %= 5; // n = n % 5 → 4
```

`n += 5` 是 `n = n + 5` 的簡寫，其他三個同理。寫起來更短，意思一樣。

比較運算子

```
int a = 10, b = 3;

cout << (a > b);    // 1 (true)
cout << (a < b);    // 0 (false)
cout << (a >= b);   // 1
cout << (a <= b);   // 0
cout << (a == b);   // 0    ← 兩個等號才是「相等」
cout << (a != b);   // 1    ← 不等於
```

比較運算子算出來的結果永遠是 `bool` (`true` 或 `false`) , `cout` 印出來會顯示成 `1` 或 `0` 。

`==` VS `=` : 最常見的錯誤

```
if (a = 5) { ... } // ❌ 這是賦值，不是比較！  
if (a == 5) { ... } // ✅ 這才是「a 是不是等於 5」
```

「`if (a = 5)`」不是在問「a 等於 5 嗎」，而是把 **5 塞進 a**，而且賦值運算式的結果永遠是 `true`（因為 5 不是 0），所以這個 if 會**永遠成立**，是很難發現的邏輯錯誤。

邏輯運算子

```
bool sunny = true, weekend = false;

cout << (sunny && weekend); // false    AND : 兩個都要成立
cout << (sunny || weekend); // true     OR  : 至少一個成立
cout << (!sunny);          // false    NOT : 反過來
```

`&&`、`||`、`!` 讓你把多個條件組合成一個。

`&&` (AND) 真值表

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

只有兩個都成立，結果才是 true。

`||` (OR) 真值表

a	b	a b
true	true	true
true	false	true
false	true	true
false	false	false

只要其中一個成立，結果就是 true。只有兩個都不成立才是 false。

實際應用：判斷分數在範圍內

```
int score = 85;  
bool valid = (score >= 60 && score <= 100); // true
```

要「同時滿足兩個條件」，就用 `&&` 把它們接起來。這裡要求 `score` 同時大於等於 60、且小於等於 100。

運算子優先順序

```
cout << 2 + 3 * 4;           // 14, 不是 20 (先乘除後加減)  
cout << (2 + 3) * 4;         // 20, 括號優先
```

跟數學一樣：先乘除後加減；比較運算子（`>`、`<`、`==`）比邏輯運算子（`&&`、`||`）先算。不確定的時候，加括號就對了——多加括號不會扣分，順序算錯才會。

if / else if / else 完整鏈

if / else if / else

```
int score = 85;

if (score >= 90) {
    cout << "A" << endl;
} else if (score >= 80) {
    cout << "B" << endl;    // ← 執行這個
} else if (score >= 60) {
    cout << "C" << endl;
} else {
    cout << "F" << endl;
}
```

由上往下檢查，第一個成立的就執行，後面全部跳過。

順序陷阱：換個順序會怎樣

```
int score = 95;

if (score >= 60) {
    cout << "C" << endl;    // ← 95 分也會走到這裡！
} else if (score >= 80) {
    cout << "B" << endl;
} else if (score >= 90) {
    cout << "A" << endl;
}
```

95 分明明應該是 A，但因為 `score >= 60` 被放在**最前面**，95 分也滿足這個條件，程式在這裡就停下來輸出 C 了，後面的判斷根本不會執行。條件要由嚴格到寬鬆排序。

逐步追蹤：為什麼順序錯了會這樣

程式檢查條件是由上往下、找到第一個成立的就停：

檢查順序	條件	score=95 成立嗎	結果
1	<code>`score >= 60`</code>	 成立	印 C，停止
2	<code>`score >= 80`</code>	不會執行到	—
3	<code>`score >= 90`</code>	不會執行到	—

正確的寫法要把**最嚴格**（``>= 90``）的條件放在最前面，這樣只有真正達到那個門檻的人才會被攔截下來。

巢狀 if : if 裡面還有 if

```
int age = 20;
bool hasTicket = true;

if (age >= 18) {
    if (hasTicket) {
        cout << "可以入場" << endl;
    } else {
        cout << "請先買票" << endl;
    }
} else {
    cout << "未成年不得入場" << endl;
}
```

「if」裡面可以再放「if」，用來表達「先滿足一個條件，才需要再檢查下一個」。今天的題目不會用到，但之後很常見，先看過一次有印象。

第 6 題 簡易計算機

輸入兩個整數與一個運算子，輸出結果。

輸入 `a op b`， $op \in \{+, -, *, /\}$

輸入 6 * 7
輸出 42

第 6 題 讀題：兩個要注意的地方

- 運算子是 ``char`` 型態，讀進來會是 ``+' '-' '*' '/'`` 其中一個
- 除法有兩個額外規則：除以 0 要輸出 ``Error``；其餘情況做整數除法

第 6 題 規劃骨架

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    char op;
    cin >> a >> op >> b;
    // 這裡依照 op 做對應運算
    return 0;
}
```

第 6 題 處理加減乘

```
if      (op == '+') cout << a + b << endl;  
else if (op == '-') cout << a - b << endl;  
else if (op == '*') cout << a * b << endl;
```

三種運算子都用字元的單引號比較：`op == '+'`，不是`op == "+"`。

第 6 題 處理除法 (含除以 0)

```
else if (op == '/') {  
    if (b == 0) cout << "Error" << endl;    // ← 別忘了!  
    else      cout << a / b << endl;        // 整數除法  
}
```

除以 0 數學上沒有定義，程式如果直接算會當掉 (RE)。一定要先檢查 `b == 0`，輸出 `Error`，其餘情況才做除法。

第 6 題 除法是整數除法

題目說「無條件捨去小數部分」。`7 / 2` 要輸出 `3`。

這題**不要**轉成 `double` ——這裡用 `int` 直接除才是正確答案，轉型反而會出錯。先讀懂題目要的是哪一種除法，再決定要不要轉型（等一下 BMI 那題就剛好相反，會需要轉型）。

第 6 題 完整程式

```
int a, b;  
char op;  
cin >> a >> op >> b;  
  
if      (op == '+') cout << a + b << endl;  
else if (op == '-') cout << a - b << endl;  
else if (op == '*') cout << a * b << endl;  
else if (op == '/') {  
    if (b == 0) cout << "Error" << endl;  
    else      cout << a / b << endl;  
}
```

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 6 題

寫到 AC 為止，再往下聽

常數與型別轉換

常數：宣告後不能再改

```
const double PI = 3.14159;  
const int MAX_SCORE = 100;
```

```
PI = 3.14;    // ❌ 編譯錯誤：不能修改常數
```

`const` 是「常數」的意思。宣告時給一次值之後，**整支程式都不能再改**。編譯器會幫你把這個規則守住，改了就報錯。

為什麼要用常數？

- 有些值本來就不該被改（例如圓周率、及格分數）
- 用 ``const`` 宣告，如果不小心手滑寫錯改到它，編譯器會馬上抓到
- 比起用一般變數，``const`` 讓別人看你的程式碼時一眼看出這是固定值

型別轉換是什麼

```
double x = 9.7;  
int y = (int)x;           // y = 9, 小數直接捨去 (不是四捨五入!)
```

`(int)x` 叫做**強制轉型**，把 `double` 硬轉成 `int`。規則很單純：**直接捨去小數點後面**，不會四捨五入。`9.7` 變成 `9`，`9.99` 也會變成 `9`。

整數除法：最常見的踩雷點

```
int a = 5, b = 2;
```

```
cout << a / b;           // 2    ← 不是 2.5 !
```

```
cout << (double)a / b;   // 2.5  ← 先轉型再除
```

$5 / 2$
兩邊都是 int
= 2

$5.0 / 2$
有一邊是小數
= 2.5

$(double)5 / 2$
明確轉型
= 2.5

為什麼 `5 / 2` 是 `2` 不是 `2.5` ？

- C++ 看到 `int / int`，會認定「你要的也是 int」
- 於是先算出真正的商 `2.5`，再把小數部分整個丟掉，只留 `2`
- 只要算式裡**有一邊是 `double`，結果就會是 `double`，才会有小數
- 這也是為什麼 `(double)a / b` 要轉的是**其中一個**，不用兩個都轉

小測驗：這樣算出來是多少？

```
int a = 7, b = 2;  
cout << a / b << endl;
```

「3」——「7 / 2」數學上是 3.5，兩邊都是 int，捨去小數變成 3。

小測驗：這樣呢？

```
int a = 7, b = 2;  
cout << (double)a / b << endl;
```

「3.5」——轉型成 double 之後再除，小數就保留下來了。

練習時間

第 7 題 BMI 分級

輸入體重（公斤）與身高（公分），計算 BMI 並分級。

$$\text{BMI} = \text{體重} \div \text{身高(公尺)}^2$$



過輕
< 18.5

正常
18.5 ~ 24

過重
24 ~ 27

肥胖
≥ 27

第 7 題 讀題：單位陷阱

輸入是公分，公式要的是公尺。而且身高讀進來是 `int` ——直接寫 `h / 100` 會得到整數 `1`（175 除以 100 捨去小數），175 公分就變成了 1 公尺，算出來的 BMI 會離譜地大。要寫 `h / 100.0`。

第 7 題 規劃骨架

```
#include <iostream>
using namespace std;

int main() {
    int w, h;
    cin >> w >> h;
    // 這裡把公分轉成公尺，算 BMI，再分級輸出
    return 0;
}
```

第 7 題 算出 BMI

```
double m = h / 100.0;           // ← 100.0 不是 100
double bmi = w / (m * m);       // w 是 int, 但除以 double 會自動轉
```

`w` 雖然是 `int`，但除以一個 `double`（`m \* m`）時會自動轉型，結果自然是 `double`，不需要額外手動轉型。

第 7 題 分級輸出

```
if      (bmi < 18.5) cout << "過輕" << endl;  
else if (bmi < 24)  cout << "正常" << endl;  
else if (bmi < 27)  cout << "過重" << endl;  
else      cout << "肥胖" << endl;
```

由上往下檢查的特性讓條件可以寫得很短：能走到第二個 `else if` 就代表 `bmi >= 18.5` 已經成立，不必再寫一次。

第 7 題 完整程式

```
int w, h;
cin >> w >> h;

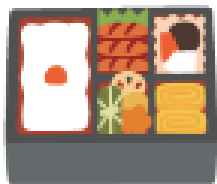
double m = h / 100.0;
double bmi = w / (m * m);

if      (bmi < 18.5) cout << "過輕" << endl;
else if (bmi < 24)  cout << "正常" << endl;
else if (bmi < 27)  cout << "過重" << endl;
else          cout << "肥胖" << endl;
```

動手做

打開 oj.itousouta.me → 課程 Day2 → 第 7 題

寫到 AC 為止——今天 6 題全部完成！



短暫休息

10 分鐘後回來

隨堂小測驗

Q1

```
int a = 5, b = 2; cout << a / b;
```

 會印出什麼？

「2」——兩邊都是 int，整數除法捨去小數。

Q2

`if (score = 90)` 這樣寫有什麼問題？

****把比較 `==` 寫成賦值 `=`，這行會把 90 塞進 `score`，而且條件永遠成立。**

Q3

``-9 % 2`` 在 C++ 是多少？

| ``-1``，不是 ``1``。判斷偶數要用 ``n % 2 == 0``，不要用 ``n % 2 == 1`` 判斷奇數。

Q4

``else if`` 的條件為什麼要**由嚴格到寬鬆**排列？

因為程式由上往下檢查，****第一個成立的就停止****。如果寬鬆的條件放前面，會提早攔截該屬於更嚴格條件的情況。

Q5

``&&`` 和 ``||`` 的差別是什麼？

``&&`` (AND) **兩個條件都要成立；``||`` (OR) **只要一個成立就夠了。

分組討論

討論①

BMI 那一題，如果拿到 ``else if (bmi < 18.5)`` 這種寫法漏掉一個邊界（例如剛好 18.5），你會怎麼設計一組測試資料去抓出這種錯誤？

討論②

跟旁邊同學交換程式碼，檢查對方的 ``if / else if`` 條件順序有沒有排錯，互相唸出每個條件的「門檻」聽聽看順序合不合理。

今日重點回顧

回顧①：型態

- 整數用 ``int``，小數用 ``double``，文字用 ``string``，單一字元用 ``char``
- ``bool`` 只有 ``true`` / ``false`` 兩種值
- 不確定用什麼型態時，先想「這個值會不會有小數點」

回顧②：變數與轉型

- 宣告就初始化，避免垃圾值
- `const` 宣告的常數不能再修改
- 整數 / 整數會捨去小數 → 要算平均、BMI 記得寫 `100.0`

回顧③：運算子

- ``%`` 餘數可以判斷奇偶、判斷倍數、取出末位數字
- 比較相等是 ``==``（兩個等號），別跟賦值 ``=`` 搞混
- ``&&`` 兩個都要成立，``||`` 一個成立就夠

回顧④：流程控制

- ``if / else if / else`` 由上往下，第一個成立的執行
- 條件順序很重要：由嚴格到寬鬆排列
- 輸出的大小寫與拼字要跟題目一模一樣

回顧⑤：今天的節奏

教了什麼	馬上練
資料型態	第 2、3 題
運算子基礎 + if 基礎	第 4 題
if 重複使用的技巧	第 5 題
運算子進階 + if/else if 完整鏈	第 6 題
常數與型別轉換	第 7 題

明天預告

明天我們用「迴圈」讓電腦幫我們**重複做**幾千幾萬次事情。

今天的 if / else 讓程式會「做一次決定」，明天的迴圈讓程式「一直做決定、一直重複」，兩個合起來威力會非常大。

謝謝大家

明天見